Portland State
Computer Science

Heavily based on material from:
Murach's MySQL (3rd Edition), by Joel Murach

What is a View?

A view is a **virtual** table based on the result-set of an SQL statement.

A view contains rows and columns, just like a *real* table. The columns in a view are columns from one or more tables in the database. Views can query from other views.

You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table.

A big difference between a view and a table is that a view does not take up space. It is a stored query, not the results from the query, just the text of the query.

A CREATE VIEW statement

```
CREATE VIEW ap.vendors_min AS
  SELECT vendor_name, vendor_state, vendor_phone
  FROM ap.vendors;
```

The virtual table that's represented by the view

	vendor_name	vendor_state	vendor_phone
▶	US Postal Service	WI	(800) 555-1205
	National Information Data Ctr	DC	(301) 555-8950
	Register of Copyrights	DC	NULL
	Jobtrak	CA	(800) 555-8725
	Newbrige Book Clubs	NJ	(800) 555-9980

(122 rows)

A SELECT statement that uses the view

```
SELECT * FROM ap.vendors_min
WHERE vendor_state = 'CA'
ORDER BY vendor_name;
```

The name of the view can be used in the from clause of a SELECT statement, just like a table name.

The result set returned by the SELECT statement

vendor_name	vendor_state	vendor_phone
Abbey Office Furnishings	CA	(559) 555-8300
American Express	CA	(800) 555-3344
ASC Signs	CA	NULL
Aztek Label	CA	(714) 555-9000
Bertelsmann Industry Svcs. Inc	CA	(805) 555-0584
BFI Industries	CA	(559) 555-1551

(75 rows)

An UPDATE statement that uses the view to update the base table

```
UPDATE ap.vendors_min
SET vendor_phone = '(800) 555-3941'
WHERE vendor_name = 'Register of Copyrights';
```

The name of a view can be used in an UPDATE statement.

A statement that drops the view

```
DROP VIEW vendors_min;
```

There are some **limitations** on how you can use views as part of an UPDATE (or INSERT or DELETE).

I happen to not like doing INSERT, UPDATE, or DELETE through a view. That may be because I've always had a choice to use any table.

Some of the benefits provided by views

- **Design independence**
  - **Views can limit the exposure of tables to external users and applications. As a result, if the design of the tables changes, you can modify the view as necessary so users who query the view don't need to be aware of the change, and applications that use the view don't need to be modified.**
- **Data security**
  - **Views can restrict access to the data in a table by using the SELECT clause to include only selected columns of a table or by using the WHERE clause to include only selected rows in a table.**
- **Simplified queries**
  - **Views can be used to hide the complexity of retrieval operations. Then, the data can be retrieved using simple SELECT statements that specify a view in the FROM clause.**
- **Updatable**
  - **With certain restrictions, views can be used to update, insert, and delete data from a base table.**

The syntax of the CREATE VIEW statement

```
CREATE [OR REPLACE] VIEW view_name
  [(column_alias_1[, column_alias_2]...)]
AS
  select_statement
```

A view of vendors that have invoices

```
CREATE VIEW ap.vendors_phone_list AS
  SELECT vendor_name, vendor_contact_last_name,
         vendor_contact_first_name, vendor_phone
  FROM ap.vendors
  WHERE vendor_id IN
     (SELECT DISTINCT vendor_id FROM ap.invoices);
```

A view that uses a join

```
CREATE OR REPLACE VIEW ap.vendor_invoices AS
  SELECT vendor_name, invoice_number, invoice_date,
         invoice_total
  FROM ap.vendors JOIN ap.invoices
    ON vendors.vendor_id = invoices.vendor_id;
```

Worn out by typing in those same old joins EVERY time?

Save common joins as views.

A view that uses a LIMIT clause

```
CREATE OR REPLACE VIEW ap.top5_invoice_totals AS
  SELECT vendor_id, invoice_total
  FROM ap.invoices
  ORDER BY invoice_total DESC
  LIMIT 5;
```

A view that names all of its columns in the CREATE VIEW clause

```
CREATE OR REPLACE VIEW ap.invoices_outstanding
  (invoice_number, invoice_date, invoice_total,
   balance_due)
AS
  SELECT invoice_number, invoice_date, invoice_total,
         invoice_total - payment_total - credit_total
  FROM ap.invoices
  WHERE invoice_total - payment_total - credit_total > 0;
```

Once you start to name the columns in a CREATE VIEW statement, you have to name

ALL of them.

A view that names just the calculated column in its SELECT clause

```
CREATE OR REPLACE VIEW ap.invoices_outstanding AS
  SELECT invoice_number, invoice_date, invoice_total,
         invoice_total - payment_total - credit_total
         AS balance due
  FROM ap.invoices
  WHERE invoice_total - payment_total - credit_total > 0;
```

A view inherits its column names from the underlying SELECT statement.

A view that summarizes invoices by vendor

```
CREATE OR REPLACE VIEW ap.invoice_summary AS
  SELECT vendor_name,
    COUNT(*) AS invoice_count,
    SUM(invoice_total) AS invoice_total_sum
  FROM ap.vendors JOIN ap.invoices
    ON vendors.vendor_id = invoices.vendor_id
  GROUP BY vendor_name;
```

Some *reporting* tables can simply be views stored in the database.

Requirements for creating updatable views

For a view to be updatable, there must be a **one-to-one relationship** between the rows in the view and the rows in the underlying table.

To create a view which is UPDATE-able, the SELECT statement that defines the view **must not contain any of the following elements**:

1. **Aggregate functions such as MIN, MAX, SUM, AVG, and COUNT.**
2. **DISTINCT**
3. **GROUP BY clause.**
4. **HAVING clause.**
5. **UNION or UNION ALL clause.**
6. Left join or outer join.
7. Subquery in the SELECT clause or in the WHERE clause that refers to the table appeared in the FROM clause.
8. Reference to non-updatable view in the FROM clause.
9. Reference only to literal values.
10. Multiple references to any column of the base table.

A CREATE VIEW statement that creates an updatable view

The `credit_total` column comes from the invoices table and has a 1-1 relationship back to that table.

```
CREATE OR REPLACE VIEW ap.balance_due_view
  SELECT vendor_name, invoice_number,
         invoice_total, payment_total, credit_total,
         invoice_total - payment_total - credit_total
          AS balance_due
  FROM ap.vendors JOIN ap.invoices
    ON vendors.vendor_id = invoices.vendor_id
  WHERE invoice_total - payment_total - credit_total > 0;
```

An UPDATE statement that uses the view

```
UPDATE ap.balance_due_view
SET credit_total = 300
WHERE invoice_number = '9982771';
```

The response from the system

```
(1 row affected)
```

An UPDATE statement that attempts to use the view to update a calculated column

```
UPDATE ap.balance_due_view
SET balance_due = 0
WHERE invoice_number = '9982771';
```

The response from the system

```
Error Code: 1348. Column 'balance_due' is not updatable
```

I find this to be an obnoxious error message. This is a reason why I don't like to update through views.

A statement that creates a view

```
CREATE VIEW ap.vendors_sw AS
SELECT *
FROM ap.vendors
WHERE vendor_state IN ('CA','AZ','NV','NM');
```

A statement that replaces the view with a new view

```
CREATE OR REPLACE VIEW ap.vendors_sw AS
SELECT *
FROM ap.vendors
WHERE vendor_state IN ('CA','AZ','NV','NM','UT','CO');
```

A statement that drops the view

```
DROP VIEW ap.vendors_sw
```

Dropping a view is less nerve racking than dropping a table. Views can be *easily* recreated (permissions restoration can be tricky).

A temporary view

- If specified, the view is created as a temporary view.
- **Temporary views are automatically dropped at the end of the current session**.
- If any of the tables referenced by the view are temporary, the view is created as a temporary view (whether TEMPORARY is specified or not).
- This is a **non-ANSI** extension to PostgreSQL.

```
CREATE TEMPORARY VIEW temp_view AS
    SELECT column1, column2
    FROM table_name
    WHERE condition;
```

```
CREATE TEMP VIEW temp_view AS
    SELECT column1, column2
    FROM table_name
    WHERE condition;
```

Materialized views

Materialized views **do persist**, but **persist the results in a table-like form**. The main differences between:

```
CREATE MATERIALIZED VIEW mymatview AS SELECT *
FROM mytab;
```

and:

```
CREATE TABLE mymatview AS SELECT * FROM mytab;
```

are that the **materialized view cannot subsequently be directly updated** and that the query used to create the materialized view is stored in exactly the same way that a view's query is stored, so that fresh data can be generated for the materialized view with:

```
REFRESH MATERIALIZED VIEW mymatview;
```

Materialized views

- The information about a materialized view in the PostgreSQL system catalogs is exactly the same as it is for a table or view.
- For the parser, a materialized view is a relation, just like a table or a view.
- When a materialized view is referenced in a query, the data is returned directly from the materialized view, like from a table; the rule is only used for populating the materialized view.
- Materialized views are a non-ANSI extension.